Laboratorio 05: Contenido sobre marcador ArUco

Guía de construcción revisada contra la escena, los prefabs y los scripts de la solución. Revisión R2, 12 de noviembre de 2026.

1. Qué vas a construir y cómo usar esta revisión

Trabaja en **Lab05-Editor-Base.zip**, abierto con Unity 6000.3.27f1. Los scripts de esta base son iguales a los de la solución. La actividad consiste en construir la escena, los prefabs y las conexiones del Inspector; no requiere escribir C#.

Si ya empezaste, conserva tu proyecto. Usa las tablas de las secciones 6, 8 y 9 para corregir conexiones. No necesitas volver a comenzar ni sustituir tus scripts para usar esta guía.

Sigue las secciones en orden. Primero crearás los objetos que se conectarán; después crearás contenido e interfaz, completarás las referencias y entrarás en Play. Un campo pendiente durante el montaje se completa antes de la sección 10.

La escena final se guarda como **Assets/Scenes/Laboratorio.unity**. La solución es una referencia del docente. Sus recursos auxiliares que no se usan en esta práctica no tienen que duplicarse para obtener el comportamiento indicado.

2. Abrir la base y comprobar versiones

1. Instala Unity 6000.3.27f1 desde Hub con Android Build Support, Android SDK & NDK Tools y OpenJDK. Extrae todo el ZIP. En Hub usa Add > Add project from disk y selecciona Lab05-Base, que contiene Assets, Packages y ProjectSettings.

2. Abre con esa versión exacta y espera la importación. Necesita internet la primera vez. No copies Library de otro proyecto ni cambies de Editor para resolver un aviso sin revisar dependencias.

3. En Window > Package Management > Package Manager, selecciona In Project. Comprueba Universal RP 17.3.0, AR Foundation 6.3.5, ARCore XR Plugin 6.3.5, XR Core Utils 2.5.1, XR Plugin Management 4.5.3, Input System 1.11.2 y Unity UI/uGUI 2.0.0. El manifest fija esas versiones.

4. Si otro equipo muestra un URP diferente, confirma primero qué carpeta y Editor abrió. Cierra Unity, guarda una copia del trabajo y compara Packages/manifest.json con el de la base. Para recuperar esta base, restaura su manifest y packages-lock.json juntos; reabre y espera la resolución. No edites Library/PackageCache ni fuerces una versión mayor de URP al azar.

5. URP 17.3.0 corresponde al Editor 6000.3.27f1 usado en esta entrega. Cambiar Editor o paquetes es una migración: hazla sobre una copia y repite las pruebas. Tener instalado URP no significa que esté asignado al proyecto.

Punto de control: no debe haber errores rojos de compilación en Console. Resuelve las versiones antes de crear prefabs o probar botones.

3. Asignar URP y revisar Android

1. Abre Assets/Settings/AR_URP. En Renderer List debe usar AR_Renderer, de tipo Universal Renderer. Revisa Rendering Path Forward y AR Background Renderer Feature en AR_Renderer. Estos recursos de apoyo ya están en la base.

2. En Edit > Project Settings > Graphics asigna AR_URP a Default Render Pipeline. En Quality revisa los niveles usados por Android: asigna el mismo asset o deja vacío el override para heredar Graphics. Un

pipeline viejo en Quality puede prevalecer sobre Graphics.

3. Si perdiste esos recursos, recupéralos de la base. Para crearlos manualmente usa Assets > Create > Rendering > URP Asset (with Universal Renderer), nombra el pipeline AR_URP y su renderer AR_Renderer y añade AR Background Renderer Feature con Add Renderer Feature. Guarda ambos en Assets/Settings.

4. Crea materiales con Universal Render Pipeline/Lit; para imágenes planas usa Universal Render Pipeline/Unlit. Si un material está magenta, revisa el shader, reasigna texturas y color. No se corrige cambiando el evento del botón.

5. En File > Build Profiles selecciona Android y Switch Platform. En Player > Other Settings conserva IL2CPP, ARM64, OpenGLES3 como única API gráfica, Minimum API 25, Target API 36 e Input System Package (New) en Active Input Handling. Acepta el reinicio si se solicita. En Optimization desactiva Strip Engine Code y conserva Assets/link.xml.

6. En XR Plug-in Management > Android comprueba ARCore e Initialize XR on Startup; revisa Project Validation. Usa Requirement Required y Depth Optional. La base incluye estos ajustes como punto de partida: debes verificarlos antes de compilar.

Punto de control: AR_URP está activo, los materiales muestran colores normales y el renderer contiene el fondo AR. Una cámara negra también puede deberse al permiso, a XR o al entorno: revisa el mensaje y Logcat antes de atribuirla a URP.

4. Crear primero la escena y sus objetos

1. Abre **Assets/Scenes/Inicio.unity**. Guarda una copia con File > Save As como **Assets/Scenes/Laboratorio.unity**. Elimina de esta escena el objeto que dice LEE LA GUIA. La base no incluye una cámara o una interfaz ya resueltas.

2. En Hierarchy, usa Create Empty y nombra el objeto **Laboratorio**. Añade los componentes **ArucoBootstrap** y **LabSceneReferences** con Add Component. Mantén Simulate In Editor activado en **ArucoBootstrap**. Este objeto ya existirá cuando conectes los botones.

3. Crea otro vacío llamado **SistemaAR**, con Position y Rotation en (0, 0, 0) y Scale (1, 1, 1). Como hijo crea GameObject > XR > AR Session. Comprueba que tenga ARSession y ARInputManager; no añadas duplicados.

4. Como otro hijo de SistemaAR crea GameObject > XR > XR Origin (Mobile AR). Nómbralo **XR Origin**. Si no aparece el atajo, crea un vacío con XROrigin, dentro un vacío **Camera Offset** y dentro una Camera llamada **AR Camera**. Conserva una sola cámara dentro del origen.

5. En XROrigin asigna Camera = AR Camera, Camera Floor Offset Object = Camera Offset y Camera Y Offset = 0. Mantén Position/Rotation en cero y Scale en uno en SistemaAR, XR Origin y Camera Offset. La cámara comienza con Position/Rotation locales en cero.

6. En AR Camera usa tag MainCamera, Near Clip 0.01, Far Clip 1000 y un AudioListener. Añade ARCameraManager y ARCameraBackground si faltan. Usa Tracked Pose Driver (Input System), no el controlador del sistema de entrada anterior.

7. En Tracked Pose Driver selecciona seguimiento de posición y rotación y deja Ignore Tracking State desactivado. Configura las acciones embebidas Position como Value/Vector3, Rotation como Value/Quaternion y Tracking State como Value/Integer. Asigna los bindings de la tabla siguiente. Si hay campos con referencias vacías, cambia a acción embebida para editar tipo y binding.

8. Desactiva los GameObjects AR Session y XR Origin con la casilla superior del Inspector. Mantén SistemaAR activo. El controlador activará la rama adecuada al iniciar. No desactives Laboratorio.

9. Crea una Directional Light llamada Luz, Rotation (45, -35, 0) e intensidad 0.9. Guarda la escena. Todavía no entres en Play: faltan contenido, interfaz y simulación.

Acción embebida	Binding principal
Position	**<HandheldARInputDevice>/devicePosition**
Rotation	**<HandheldARInputDevice>/deviceRotation**
Tracking State	**<HandheldARInputDevice>/trackingState**

Puedes añadir los bindings alternos **<XRHMD>/centerEyePosition**, **<XRHMD>/centerEyeRotation** y **<XRHMD>/trackingState** a las acciones correspondientes. No reemplazan los bindings del dispositivo AR.

5. Crear y asignar los recursos de esta práctica

En Project crea las carpetas **Assets/Prefabs**, **Assets/Materiales** y **Assets/Configuracion** si faltan.

Crear el catálogo ContenidoLab

1. En la ventana Project, abre Assets/Configuracion. Haz clic derecho en el espacio de esa carpeta y elige Create > Laboratorios > Contenido. También puedes usar el menú superior Assets > Create > Laboratorios > Contenido. El nombre que debes buscar en el menú es Contenido.

2. Unity creará un asset llamado ContenidoLab. Conserva ese nombre. LabContent es el tipo del asset; ContenidoLab.asset es el archivo que creas para guardar las referencias de esta práctica.

3. Selecciona ContenidoLab.asset en Project. Su Inspector debe mostrar Content A, Content B, Indicator, Capture Target, Projectile, Aruco Visual y Selected Material. Completarás únicamente los campos indicados para esta práctica en los pasos siguientes y en la tabla de la sección 8.

4. El script Assets/Scripts/Course/LabContent.cs ya está incluido y hereda de ScriptableObject. No se agrega a Laboratorio con Add Component. Cuando llegues a la sección 8, arrastra ContenidoLab.asset al campo Content del controlador que sí está en Laboratorio.

5. Si no aparece Create > Laboratorios > Contenido, espera a que termine la importación y comprueba que LabContent.cs exista en la ruta anterior. Abre Window > General > Console y resuelve el primer error rojo de compilación. No crees otro LabContent.cs ni copies archivos de la solución para hacer aparecer el menú. Si sigue sin aparecer después de compilar sin errores, conserva una captura del menú y el mensaje de Console para revisar el caso.

Construir los prefabs del catálogo

Cuando un paso indique asignar un prefab al catálogo, arrastra el asset de Project, no una instancia temporal de Hierarchy.

Crea los materiales en Assets/Materiales con shader Universal Render Pipeline/Lit, salvo los materiales de imágenes que indican Unlit. El color de un objeto se asigna arrastrando el material a su MeshRenderer; nombrar un material no lo conecta automáticamente.

Para guardar un prefab, arrastra su raíz de Hierarchy a Assets/Prefabs. Si lo editas en Prefab Mode, guarda antes de cerrarlo. Elimina las instancias de montaje una vez guardadas: el controlador creará las figuras desde el catálogo.

Para el contenido ArUco, normaliza la malla a una unidad y conserva la raíz en Scale (1, 1, 1). La lógica aplica el tamaño final al instanciarlo.

1. Crea un material naranja URP/Lit. Crea un vacío ContenidoArUco con un cubo hijo centrado, normalizado a una unidad. Desactiva o elimina colliders de este contenido visual. Guarda ContenidoArUco.prefab y elimina su instancia de montaje. Puedes sustituir la malla por otro modelo bajo esa raíz.
2. En ContenidoLab asigna Aruco Visual = ContenidoArUco.prefab. El resto queda vacío. La lógica lo muestra a 5 cm y eleva su centro 2.5 cm sobre el marcador.
3. Usa DICT_4X4_50 ID23 de Marcadores. Este detector didáctico reconoce solo ese ID por coincidencia exacta: no implementa todo OpenCV ni corrección de errores. No añadas Reference Image Library ni ARTrackedImageManager.
4. Imprime Marcador-Lab05-ArUco23-10cm.pdf al 100 %. Mide el cuadrado negro exterior sin margen blanco: 10 cm. En ArucoBootstrap configura Marker Side Meters = 0.10. Si cambia el tamaño físico, escribe el lado real en metros y recompila.
5. En SimulationFrame.png activa Read/Write, desactiva mipmaps, usa Compression None, Non Power of 2 = None y Max Size 1024. La lógica procesa las dimensiones reales importadas; deformar la textura cambia la geometría de prueba.

Antes de continuar, abre ContenidoLab y verifica las asignaciones de esta sección. Guarda los prefabs y elimina sus instancias de montaje. Los objetos que deben quedar en escena se muestran en la sección 9.

6. Construir la interfaz y conectar cada botón

1. Selecciona la raíz de Hierarchy y crea GameObject > UI > Canvas. Nómbralo **Interfaz**. Añade **LabUI** y conserva CanvasScaler y GraphicRaycaster. Usa Canvas Render Mode = **Screen Space - Overlay**. No añadas una Image que cubra todo el Canvas.
2. En CanvasScaler usa Scale With Screen Size, Reference Resolution (540, 960), Screen Match Mode = Match Width Or Height y Match = 0.5. No cambies el RectTransform del Canvas para posicionar los controles.
3. Crea un Panel como hijo directo de Interfaz y nómbralo **PanelEstado**. En RectTransform usa Anchor Min y Max (0.5, 1), Pivot (0.5, 0.5), Anchored Position (0, -90) y tamaño (516, 130). Asigna un fondo oscuro semitransparente. Solo la cabecera debe ocupar ese rectángulo.
4. Dentro de PanelEstado crea UI > Legacy > Text y nómbralo **Estado**. Estíralo con Anchor Min (0, 0), Max (1, 1), Pivot (0.5, 0.5) y márgenes Left/Right/Top/Bottom de 10. Usa texto blanco, alineación central y tamaño de fuente 21. Desactiva Raycast Target del Text. Arrastra su componente Text al campo Status de LabUI.
5. Crea los botones indicados en la tabla como hijos directos de Interfaz. No los dejes dentro de PanelEstado: arrástralos fuera si Unity los creó bajo el objeto seleccionado. Usa UI > Legacy > Button para que cada uno tenga Button y un Text hijo, compatibles con LabUI.
6. En el RectTransform de cada botón fija Anchor Min y Max (0.5, 0), Pivot (0.5, 0.5), X = 0, Width = 480 y Height = 58. Usa la Y indicada en la tabla. Conserva Raycast Target en su Image y desactívalo en el Text hijo. Escribe una etiqueta inicial legible; LabUI actualizará las etiquetas y estados durante Play.
7. En LabUI asigna cada componente Button al campo indicado. Estas referencias permiten al script actualizar las etiquetas y habilitar o deshabilitar controles. No sustituyen las conexiones de On Click ().
8. Selecciona cada botón. En Button > On Click () pulsa +, usa Runtime Only y arrastra el GameObject Laboratorio de Hierarchy. En la función elige ArucoBootstrap y el método de la tabla, sin argumentos. Debe haber una sola entrada por botón. El destino es el objeto con el componente, no el archivo C# ni el prefab de contenido.

9. Conserva un solo EventSystem activo en la escena. Si Unity lo creó al añadir Canvas, usa ese. Elimina StandaloneInputModule si aparece y añade InputSystemUIInputModule. En Actions Asset asigna Assets/Input/AccionesUI.asset y completa las acciones de la segunda tabla. Expande el asset en Project para arrastrar sus subassets.
10. Guarda la escena. Si guardas Interfaz como prefab, conserva las conexiones On Click como overrides de su instancia en esta escena. Un prefab de interfaz no puede almacenar una referencia externa al Laboratorio de la escena.

Botón bajo Interfaz	Y	Campo de LabUI	Método en Laboratorio
Accion1	245	Primary	**ArucoBootstrap.CornersButton**
Accion2	175	Secondary	**ArucoBootstrap.FlipButton**
SimularPerdida	105	Simulation	**ArucoBootstrap.TrackingButton**

Campo de InputSystemUIInputModule	Subasset de AccionesUI
Point	UI_Point
Left Click	UI_Click
Move	UI_Navigate
Submit	UI_Submit
Cancel	UI_Cancel
Scroll Wheel	UI_ScrollWheel
Right Click	UI_RightClick
Middle Click	UI_MiddleClick

En LabUI deja Tertiary y Quaternary vacíos. No se usan botones Eliminar o Superficies en esta práctica.

Crea cuatro UI > Image como hijos directos de Interfaz, llamados Esquina0 a Esquina3. Usa tamaño (10, 10), color verde y Raycast Target desactivado. En LabUI, Corner Dots debe tener Size = 4; asigna el RectTransform de cada Image a un elemento distinto. No los coloques dentro de PanelEstado porque el script los posiciona por coordenadas de viewport.

7. Construir el entorno de simulación

1. Crea Simulacion con CamaraSimulada, Position/Rotation 0, tag MainCamera, Field of View 60, near clip 0.01 y AudioListener. Usa Game en relación 4:3. Crea un Quad ImagenEstatica en (0, 0, 0.18895), escala aproximada (0.29091, 0.21818, 1), collider desactivado y URP/Unlit con SimulationFrame.png.
2. Asigna Simulation World y Simulation Camera en LabSceneReferences, guarda Simulacion.prefab y desactiva su raíz. No dejes ambas cámaras activas simultáneamente.

Crea un material con shader Universal Render Pipeline/Unlit, asigna Assets/Resources/SimulationFrame.png a Base Map y arrastra el material al MeshRenderer de ImagenEstatica. El Quad se conserva en escena; el contenido ArUco se instanciará desde el catálogo.

Crea todos los objetos de esta sección dentro de la raíz Simulacion. Introduce posiciones y rotaciones como valores locales de sus Transform, con la raíz en Position/Rotation cero y Scale uno. Mantén Simulacion desactivado al guardar; el controlador lo activará en Editor.

8. Completar componentes AR y referencias finales

1. Selecciona AR Camera y añade CpuGrayCamera una sola vez. ArucoBootstrap obtiene este componente de esa cámara al iniciar; no lo pongas en Laboratorio ni en CamaraSimulada.
2. En ArucoBootstrap conserva Simulate In Editor activo, Marker Side Meters = 0.10 y Flip Cpu Image Y desactivado inicialmente. No añadas ARTrackedImageManager ni administradores de planos. Esta práctica procesa la imagen CPU y no utiliza una Reference Image Library.

Ahora que todos los objetos existen, completa la tabla. Para referencias de escena arrastra desde Hierarchy; para Content y los recursos del catálogo arrastra desde Project. Un campo None en una fila requerida debe resolverse antes de Play.

Objeto y componente	Campo	Referencia requerida
Laboratorio / ArucoBootstrap	Scene	LabSceneReferences del mismo Laboratorio
Laboratorio / ArucoBootstrap	Content	Assets/Configuracion/ContenidoLab.asset
Laboratorio / ArucoBootstrap	UI	LabUI del Canvas Interfaz
Laboratorio / LabSceneReferences	Ar Origin	GameObject XR Origin de esta escena
Laboratorio / LabSceneReferences	Ar Session	GameObject AR Session de esta escena
Laboratorio / LabSceneReferences	Ar Camera	Componente Camera de AR Camera
Laboratorio / LabSceneReferences	Simulation World	GameObject Simulacion de esta escena
Laboratorio / LabSceneReferences	Simulation Camera	Componente Camera de CamaraSimulada

Abre ContenidoLab.asset y verifica también este catálogo:

Campo de ContenidoLab	Asset requerido
Aruco Visual	ContenidoArUco.prefab

Los campos de LabSceneReferences y ContenidoLab no incluidos en estas tablas no se utilizan en esta práctica y pueden quedar vacíos. Los campos visibles extra pertenecen a los otros laboratorios; no necesitas añadir sus componentes.

9. Revisar jerarquía, estados y conexiones

Comprueba esta estructura antes de iniciar. Los nombres de las mallas o materiales propios pueden cambiar; las relaciones padre/hijo y las referencias deben corresponder a lo indicado. EventSystem también puede estar dentro de Interfaz, pero debe existir solo uno.

```
Laboratorio
Luz
SistemaAR
  AR Session
  XR Origin
    Camera Offset
      AR Camera
Interfaz
  PanelEstado
    Estado
  Accion1
  Accion2
  SimularPerdida
  Esquina0 ... Esquina3
EventSystem
Simulacion
  CamaraSimulada
  ImagenEstatica
```

Objeto	Estado al guardar, fuera de Play
Laboratorio	Activo
SistemaAR	Activo
AR Session	Inactivo
XR Origin	Inactivo
AR Camera	Activo por sí mismo, bajo XR Origin inactivo
Simulacion	Inactivo
CamaraSimulada	Activo por sí mismo, bajo Simulacion inactivo
Interfaz y EventSystem	Activos

Guarda SistemaAR y Simulacion como prefabs, conservando sus instancias en esta escena. Guarda Interfaz si quieres reutilizar su estructura; los destinos de On Click permanecen en la instancia de Laboratorio de esta escena.

Ejecuta Tools > Laboratorios > Verificar escena (sin modificar) y resuelve los errores. Ese verificador es parcial: su mensaje CONFIGURACION CORRECTA no comprueba todas las referencias de los managers, los botones requeridos ni que cada callback sea el esperado. Revisa manualmente todas las filas de las secciones 6 y 8 aunque no reporte errores.

Comprueba que no queden instancias de montaje de los contenidos en Hierarchy. Las figuras aparecerán al ejecutar la lógica, no como decoración ya colocada. No pruebes un prefab aislado en Prefab Mode: abre Laboratorio.unity para probar todas las conexiones.

10. Probar el resultado en el Editor

1. En Play debe informar ID23 reconocido y mostrar el prefab. El botón de puntos debe cambiar una sola vez por clic. Simular pérdida oculta el objeto tras 0.35 s y Volver a detectar lo recupera. Esta prueba no usa cámara física.

No continúes a la compilación mientras haya errores rojos o falle una acción de esta sección en simulación. El resultado de Editor comprueba la escena y la interacción; no valida por sí solo la cámara, los permisos y el rastreo físicos de Android. Las pruebas con el celular están después de la compilación, en la sección 12.

11. Cambiar el modelo después de completar la prueba

1. Importa tu FBX u OBJ en Assets/Modelos. También se incluye PiramideEjemplo.obj. Revisa escala y orientación en el importador. Un OBJ puede traer solo geometría: crea y asigna materiales URP a sus renderers.
2. Guarda el prefab. Los cambios de escala hechos solo en una instancia durante Play se pierden al detenerlo. Una raíz normalizada facilita reemplazar modelos conservando el tamaño y el contacto con el plano.
3. En esta práctica, asigna el nuevo prefab a Aruco Visual de ContenidoLab. Su raíz debe estar centrada y normalizada a una unidad. Se muestra a 5 cm, con su centro 2.5 cm sobre el marcador. Desactiva colliders de este contenido decorativo. El tamaño físico del marcador sigue siendo 10 cm.
4. Detén Play, cambia el prefab o la referencia y vuelve a ejecutar. Vuelve a detectar el marcador para comparar el contenido.

12. Compilar y probar Android

1. Sal de Play y guarda escena y prefabs. Ejecuta Tools > Laboratorios > Verificar escena (sin modificar). Resuelve lo informado. En Build Profiles deja habilitada solo Assets/Scenes/Laboratorio.unity y retira Inicio.unity de la lista final.
2. Revisa IL2CPP/ARM64, API 25-36, OpenGLES3, Input System (New), ARCore, link.xml y Strip Engine Code desactivado. No borres .meta ni cambies GUID manualmente para reparar referencias.
3. En Windows usa Tools > Laboratorio > Compilar APK Android. El comando compila las escenas habilitadas mediante el flujo directo de Unity, con su SDK, NDK, JDK y Gradle. El APK y su registro .build.txt quedan en la carpeta APK junto a la raíz del proyecto. Confirma LAB_BUILD_SUCCEEDED y el resultado Succeeded antes de instalarlo.
4. El flujo Tools > Laboratorio > Exportar y compilar con Gradle (alternativo) conserva la exportación para quien la necesite. Usa primero el comando directo indicado en el paso 3.
5. Build And Run también sirve cuando ese flujo compila en tu entorno. Conecta un Android con ARCore, desbloquea, autoriza depuración USB y concede cámara. Una firma diferente puede impedir reemplazar un APK previo: conserva evidencias antes de desinstalar porque puedes perder guardado local.
6. Haz la prueba física indicada a continuación. Ante un fallo registra el mensaje exacto de Console/Android Logcat. Revisa primero permisos, inicialización y rastreo; después acción del botón y referencias a prefabs.

Prueba física después de instalar

1. En Android deja visibles cuadrado negro y margen blanco. Primero comprueba los cuatro puntos verdes sobre las esquinas reales. Si están reflejados, prueba Invertir Y. Si ninguna opción alinea, revisa la conversión de imagen a viewport y la orientación; no compenses moviendo el prefab.
2. Con las esquinas alineadas, mueve y gira la hoja: el contenido debe seguirla. Tapa y recupera el marcador. Registra distancia, iluminación e inclinación donde falla. Una coincidencia exacta puede perder detección con ruido, desenfoque, reflejos u oclusión.

3. Invertir Y es temporal. Si el equipo necesita el otro valor, guárdalo en Flip Cpu Image Y y recompila. El contenido no permanece sin el marcador ni se guarda entre sesiones.

13. Evidencias y diagnóstico por etapa

Entrega Assets, Packages y ProjectSettings de tu proyecto, el APK compilado y un video de las pruebas físicas de la sección 12. Añade capturas del catálogo, las referencias de Laboratorio, los callbacks On Click, EventSystem y la configuración URP. Excluye Library, Logs, Temp y Build/AndroidGradle.

Registra el modelo de celular y el mensaje exacto de Console o Android Logcat cuando falle una prueba. Revisa primero la inicialización, las referencias y los permisos; después distingue si falla el botón, la detección o la colocación. Una compilación sin errores no demuestra que el flujo físico completo funcione.

Si el botón se dibuja pero no responde, revisa EventSystem, InputSystemUIInputModule y sus acciones, GraphicRaycaster y On Click. Las imágenes decorativas y los textos deben tener Raycast Target desactivado; los botones deben conservarlo en su Image. No añadas OnGUI ni otra lectura manual para disparar la misma acción. En esta edición el código excluye los toques sobre UI antes de colocar contenido en el mundo.

Invertir Y está desactivado en simulación. La imagen estática no demuestra alineación con la cámara física. Antes de evaluar la posición del modelo en Android, alinea los cuatro puntos verdes con las esquinas reales del marcador.

Fuentes técnicas de consulta:

- Input System 1.11 y uGUI: https://docs.unity3d.com/Packages/com.unity.inputsystem@1.11/manual/UISupport.html

- AR Foundation 6.3: https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.3/manual/index.html

- Pipeline activo en Graphics y Quality: https://docs.unity3d.com/6000.1/Documentation/Manual/srp-setting-render-pipeline-asset.html